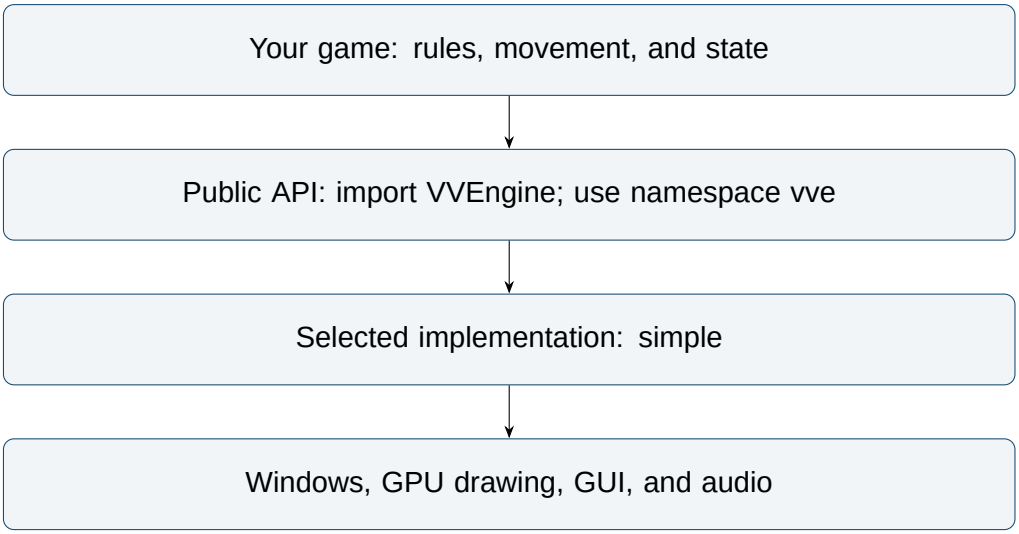
VIENNA VULKAN ENGINE

Your first VVE program

A short guide for a first project

You need basic C++: variables, functions, and structs. You do not need to know Vulkan, shaders, or how a game engine works. Start with a supplied example, then build a small program of your own.



Updated 16 October 2026

Source baseline: 8f875a30; API checked against the local checkout.

Game code uses the facade. Implementation chapters are for source study.

A route through this guide

Read sections 1–4 to build the engine and show a scene. Section 5 adds code that runs every frame. The remaining sections explain input, models, the scene tree, GUI images, sound, and common failures. The longer companion, `simple_engine_explained.pdf`, explains each idea more slowly and then follows the implementation.

VVE means Vienna Vulkan Engine. Your program calls a public interface called the *facade*. The engine implementation handles the platform and GPU work behind that interface. Write `import VVEngine;` and use names beginning with `vve::`. Link the engine's CMake target. Both steps are necessary.

1 Build a supplied example first

Work from the repository root: the folder containing `README.md`, `src`, and the build scripts. The first build installs dependencies and may take much longer than later builds. A dependency is another library the engine needs.

Windows: PowerShell

Install Git, Visual Studio's *Desktop development with C++* workload and CMake tools, a suitable CMake, and the Vulkan SDK. The repository documents CMake 3.31.8 or newer; its `import std` support must match the selected compiler and CMake version. Install vcpkg, run its bootstrap script, and set `VCPKG_ROOT` to its folder in the shell you use to build.

```
$env:VCPKG_ROOT = 'C:\tools\vcpkg'
.\build_windows.cmd debug
.\bin\debug\exe\simple_forward_demo.exe --frames 120
```

The build script sets up the Visual Studio compiler. When using Windows CMake presets directly, use a Developer PowerShell for Visual Studio.

Linux

Install Ninja and matching Clang, clang-scan-deps, and libc++ standard-library modules. The script selects a complete installed LLVM toolchain. Install the Vulkan SDK and the vcpkg dependencies described in the README.

```
./build_linux.sh debug
./bin/debug/exe/simple_forward_demo --frames 120
```

macOS

Use the documented LLVM and Ninja setup, the Vulkan SDK, and KosmicKrisp. Follow the README's loader-manifest setup for your build directory.

```
./build_macos.sh debug
./bin/debug/exe/simple_forward_demo --frames 120
```

Remember. A compiler turns C++ into a program. CMake describes what to build. Ninja runs the build commands. vcpkg installs libraries. The Vulkan SDK supplies the Vulkan tools and Slang shader compiler.

Source: `README.md;build_linux.sh;build_macos.sh;build_windows.cmd`.

2 Find the output and understand a failure

A successful example opens a window and renders a scene. The `--frames 120` option makes this supplied example stop after a bounded number of steps. This option is handled by example code; a new program does not automatically understand it.

Debug executables are in `bin/debug/exe`; release executables are in `bin/release/exe`. Windows adds `.exe`. Run from the repository root while learning, so paths such as `assets/...` start in the expected folder. A relative filename is relative to the process's working directory, not necessarily the executable's directory.

The build scripts also run tests. Distinguish these stages:

1. Configuration finds tools and libraries.
2. Compilation checks C++ and builds targets.
3. Linking joins the compiled pieces.
4. Tests run the resulting programs and check their behavior.

Read the first meaningful error in the failing stage. Changing game code will not repair a missing compiler or unsupported graphics driver.

Useful supplied programs include `game` for a small game, `hierarchy_car` for connected parts, `audio_demo` for sound, `postprocessing` for image effects, and `light_shadow_debug` for a PNG and numerical shadow diagnostics. The `sponza` target currently loads the bundled castle scene; the target name does not describe the current asset.

> **Try it.** Build and run one example without changing its source. Record the command and working directory that succeeded. This gives you a known working starting point.

3 Add a small program to the build

Create `examples/mygame/main.cpp`. Put the complete program from the next section in it. Create `examples/mygame/CMakeLists.txt`:

```
add_executable(mygame main.cpp)
target_link_libraries(mygame PRIVATE
    ViennaVulkanEngine::ViennaVulkanEngine)
vve_set_output_dirs(mygame)
```

Add `add_subdirectory(mygame)` once to `examples/CMakeLists.txt`, then rerun the normal build script. Launch `bin/debug/exe/mygame` on Linux/macOS or `.\bin \debug \exe \mygame .exe` on Windows.

This layout inherits the repository's C++23 module setup. Do not include an `.ixx` file as a header or copy compiler-generated module files by hand. A separate parent CMake project requires additional `import std` setup before `project()`; use the README when you reach that stage.

Source: `src/CMakeLists.txt;examples/CMakeLists.txt;README.md`.

4 A complete first program

This program uses a built-in sample scene. It needs no model or texture file. Copy the whole listing into `main.cpp`.

```
import std;
import VVEngine;

int main() {
    auto engine = vve::EngineBuilder<>{}
        .applicationName("My first VVE program")
        .addWindow(vve::WindowSetup{}
            .id("main")
            .title("Hello VVE")
            .extent(vve::PixelExtent{800, 600}))
        .build();

    if (const auto result = engine.init(); !result) {
        std::cerr << vve::errorName(result.error()) << '\n';
        return 1;
    }
    auto world = engine.world();
    auto &render = world.get<vve::RenderSystem>();
    if (const auto result = render.loadSampleScene(); !result) {
        std::cerr << vve::errorName(result.error()) << '\n';
        return 2;
    }
    if (const auto result = engine.run(); !result) {
        std::cerr << vve::errorName(result.error()) << '\n';
        return 3;
    }
    return 0;
}
```

Expected result: a window containing the sample objects, lit by the sample lights. Close the window to leave `run()`.

`EngineBuilder<>` collects settings; `build()` constructs the engine. The empty angle brackets mean that no game systems are registered yet. `init()` creates the platform windows. GPU rendering begins lazily when a drawable window reaches its first render.

`world.get<vve::RenderSystem>()` returns a reference to this engine's renderer. The `&` in `auto &render` means that the variable refers to the existing object. The engine must stay alive while you use that reference.

A fallible call returns `std::expected`: either a value or an error. `!result` asks whether it failed. Only use `*result` after checking that it succeeded. The first two `import` lines make standard-library and VVE declarations available.

Source: `src/Engine.ixx;src/World.ixx;src/RenderSystem.ixx`.

5 A game system runs your update code

A *frame* is one trip through the main loop. A *system* is an ordinary C++ object with methods the engine recognizes. Replace `main.cpp` with this complete program.

```
import std;
import VVEngine;

struct Game {
    double elapsed{0.0}; // This value survives between frames.

    auto init(auto &world) -> std::expected<void, vve::Error> {
        return world.template get<vve::RenderSystem>()
            .loadSampleScene();
    }

    void update(auto &world, const vve::FrameContext &frame) {
        elapsed += frame.delta_time.seconds;
        auto &windows = world.template get<vve::WindowSystem>();
        auto input = windows.input();
        if (input.wasKeyPressed(vve::Key::p)) {
            std::cout << "P pressed at " << elapsed << " seconds\n";
        }
    }
};

int main() {
    vve::Engine<Game> engine;
    if (const auto result = engine.run(); !result) {
        std::cerr << vve::errorName(result.error()) << '\n';
        return 1;
    }
    return 0;
}
```

The engine owns one `Game`. It calls `init` during startup and `update` after input polling and before rendering. Press P in the focused window to print a message. A key press is a transition; holding P does not repeatedly trigger this branch every frame.

The type of `world` is generic because the function uses `auto &`. C++ therefore needs `template` in `world.template get<...>()`. You can copy that spelling without knowing how templates are implemented.

`frame.delta_time.seconds` measures elapsed time. Move by *speed multiplied by elapsed seconds*, rather than a fixed distance per frame. The first delta uses `DeltaTime{}`: currently 1/60 second.

Source: `src/Engine.ixx;src/Types.ixx;src/Window.ixx`.

6 Make an object move

Use `RenderObjectHandle` to remember a visible object. A handle is a typed ID, not the object itself. Store it as a member of your system, alongside the position you want.

The following are **excerpts for a Game system**, not a second complete program. In `init`, obtain the renderer, create the cube, and remember its handle:

```
auto &render = world.template get<vve::RenderSystem>();
auto cube = render.addCuboid(
    vve::Vec3{-0.5F, -0.5F, -0.5F},
    vve::Vec3{ 0.5F,  0.5F,  0.5F},
    vve::LinearColor{.value = vve::Vec3{0.2F, 0.7F, 0.3F}});
if (!cube) { return std::unexpected(cube.error()); }
object = *cube; // Game member: vve::RenderObjectHandle object{};
```

For an immediately visible teaching object, call `setObjectUnlit(object, true)` and check its result. An unlit object shows its colour without needing lights. Set a camera looking at it, or start from the complete moving-cube program in the longer guide.

In `update`, obtain input, change your stored transform, and submit the new transform. This version returns an error, so declare the hook to return `std::expected<void, vve::Error>`:

```
auto input = world.template get<vve::WindowSystem>().input();
const float dt = static_cast<float>(frame.delta_time.seconds);
if (input.isKeyDown(vve::Key::d)) {
    pose.translation.value.x += 2.0F * dt;
}
// Game member: vve::Transform pose{};
return world.template get<vve::RenderSystem>()
    .setObjectTransform(object, pose);
```

At speed 2, a one-second interval produces two world units of movement. VVE does not attach a real-world unit to a coordinate; using one unit as one metre is a useful game convention.

7 Camera, colour, and the background

A camera is the view of the scene. These calls belong after obtaining `render`:

```
render.setCamera(vve::Camera::lookAt(
    vve::Position{.value = vve::Vec3{0, 3, 7}},
    vve::Position{.value = vve::Vec3{0, 0, 0}}));
render.setBackgroundColor(vve::LinearColor{
    .value = vve::Vec3{0.08F, 0.12F, 0.18F}});
```

The sample coordinates use Y as up. A default camera looks along negative Z. Its clipping distances default to 0.1 and 100 world units; objects outside that visible depth interval are clipped. The full guide explains how to fit a camera to a large model.

Source: `src/RenderSystem.ixx;src/Types.ixx`.

8 Load a model and move its parts

A *model* is a file containing geometry and often materials and a tree of named parts. Loading it makes source data available. Instantiating it creates visible placements. These are separate steps.

This excerpt assumes `world` exists and `modelPath` is a `std::filesystem::path` naming a real model:

```
auto &assets = world.get<vve::AssetSystem>();
auto &render = world.get<vve::RenderSystem>();
auto scene = assets.loadScene(modelPath);
if (!scene) { return 1; }
auto instance = render.instantiateScene(*scene,
    vve::SceneInstantiationOptions{.keep_hierarchy = true});
if (!instance) { return 2; }
auto root = render.sceneInstanceRoot(*instance);
if (!root) { return 3; }
vve::Transform placement{};
placement.translation.value.x = 3.0F;
if (!render.setNodeTransform(*root, placement)) { return 4; }
```

With `keep_hierarchy`, the engine copies the model's tree into the world scene tree. Moving the instance root moves the entire model. A wheel node can then move relative to the car. Use `sceneInstanceNode` to find the runtime copy of a particular asset node. `NodeHandle` and `SceneNodeHandle` identify different kinds of node.

Imported lights and cameras are not applied by default. Set your own camera and light, or deliberately enable `apply_lights` and `apply_cameras` in the instantiation options. An imported model can also be very large or far from the origin; adjust the camera before assuming the import failed.

9 Textures, sky, sprites, and picking

- `addTexturedPlane({20,20}, texturePath, {10,10})` creates a 40 by 40 XZ ground plane, repeating the image ten times along each axis.
- `setSkybox(panoramaPath)` shows a 2:1 equirectangular panorama around the camera. Check the returned result.
- `addSprite(size, texturePath)` creates a flat image that turns toward the camera. Transparent pixels allow shaped icons or particles.
- `screenRay(window, normalizedPoint)` builds a ray through a screen location. `raycast(origin, direction, distance)` finds the nearest visible object's bounding box along it.

These are separate operations. A sprite is a world object; a GUI image is drawn on the interface. The full guide supplies examples and explains opacity and box-based picking.

Source: `src/Assets.ixx;src/RenderSystem.ixx;examples/hierarchy_car/hierarchy_car.cpp`.

10 Add an interface and optional sound

A GUI callback

Include `<imgui.h>` before the module imports. Register a callback once; it builds the interface each GUI frame:

```
// Inside Game::init; Game has an int score member.
auto &gui = world.template get<vve::GuiSystem>();
gui.draw([this] {
    ImGui::Begin("Score");
    ImGui::Text("Collected: %d", score);
    ImGui::End();
});
```

For a HUD image, call `gui.loadImage(path)`, check the returned handle, and retain it. Inside the callback call `gui.imageId(handle)`. Draw only when the ID is nonzero: the image may still be waiting for GPU upload. The full guide has a complete recipe.

Sound resources and playing voices

Audio is optional and opens separately from graphics. This excerpt assumes `musicPath` names an existing audio file:

```
auto &audio = world.get<vve::AudioSystem>();
if (!audio.init()) {
    std::cerr << audio.lastError() << '\n';
    return 1;
}
auto sound = audio.loadSound(musicPath,
    vve::AudioLoadMode::on_demand);
if (!sound) { return 2; }
auto voice = audio.play(*sound,
    vve::AudioPlaybackOptions{.loop = true,
        .volume = vve::AudioVolume{.value = 0.4F}});
if (!voice) { return 3; }
```

A sound handle identifies reusable loaded data. A playback handle identifies one playing voice. Two `play` calls can overlap. Keep `*voice` for `pause`, `resume`, and `stop`; check those calls too. Gain is between 0 and 1. `on_demand` decodes during playback; it is not disk streaming.

The engine must stay alive during playback. Finished voices are collected during engine steps and before another play call. `docs/Audio.md` explains the audio contract and the bundled examples.

Source: `src/Gui.ixx;src/Audio.ixx;docs/Audio.md`.

11 Troubleshoot one stage at a time

Symptom	First useful check
Compiler or module error	Use the documented compiler, Ninja, and matching standard-library modules. Rebuild through the normal script.
Missing dependency	Check the Vulkan SDK and vcpkg installation, triplet, and environment. Read the configure error.
No suitable GPU	Read the rejected-device diagnostic. The simple renderer checks Vulkan 1.3 device features and requests a Vulkan 1.4 instance.
Window but no object	Confirm creation and instantiation succeeded. Check visibility, camera, clipping distances, object position, and lighting.
File not found	Print the working directory and exact path. Relative paths start at the working directory.
Texture remains broken after repair	The simple engine caches decode failures. Clear the scene before retrying that repaired source.
No sound	Call `audio.init()`; inspect `lastError()`, file paths, voice state, and gain.
Input seems ignored	Focus the window. An active ImGui widget can claim input. Use held-key queries for continuous movement.
Test failure	Read the named CTest result and its output. Build success and test success are different checks.

Run tests again with `ctest --test-dir <build-directory> --output-on-failure`. Linux script builds use `build/debug-linux`; macOS script builds use `build/macos-debug`; Windows script builds use `build/debug-windows`. Preset and VS Code build directories can differ.

Where to go next

Read the longer guide for a complete moving-cube project, ECS-to-render synchronization, local versus world transforms, multiple windows, lighting, GUI images, resource lifetimes, and the CPU-to-GPU path. Use `README.md` for platform prerequisites and `tests/CMakeLists.txt` for the tests actually registered in your checkout.

About this edition

Updated for the facade and simple engine at source baseline `8f875a30`, including current procedural shapes, appearance controls, skyboxes, sprites, picking, and HUD images. The previous engine analysis built the Linux release targets and passed 80 registered tests. That is a dated result, not a promise about future checkouts. New guide examples are checked separately during this documentation update. Concurrent racer work is outside this edition.

Source: `README.md;src/Engine.ixx;src/Types.ixx;tests/CMakeLists.txt`.